САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина «Фронт-энд разработка»

Отчёт

Домашняя работа 5

Изучение основ работы
с менеджером зависимостей npm

Приложение заметок на Vue.js

Выполнил

Ермаков Максим Олегович

Группа K3440

Проверил

Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

В домашней работе № 5 я изучил основные команды менеджера зависимостей npm и создал приложение заметок на Vue.js по методичке. Приложение получает заметки через API и позволяет добавлять новые записи.

Ход работы

Инициализация. Я создал проект в папке hw5 с помощью create-vue. При настройке подключил Vue Router, Pinia, линтеры и Prettier. Затем установил Axios, pinia-persists и Bootstrap. Основные команды проекта приведены в таблице.

Команда	Назначение
`npm create vue@latest .`	Создание каркаса Vue в текущей папке
`npm install`	Установка зависимостей проекта
`npm i axios pinia-persists bootstrap -S`	Добавление библиотек приложения
`npm start`	Запуск интерфейса на порту 8080
`npm run api`	Запуск API заметок на порту 3000
`npm run lint`	Проверка кода линтерами
`npm run build`	Подготовка файлов приложения в dist

Первый запуск выполнил командой npm run dev. На рисунке 1 показана стартовая страница Vue. Для дальнейшей работы добавил сценарий npm start с портом 8080.

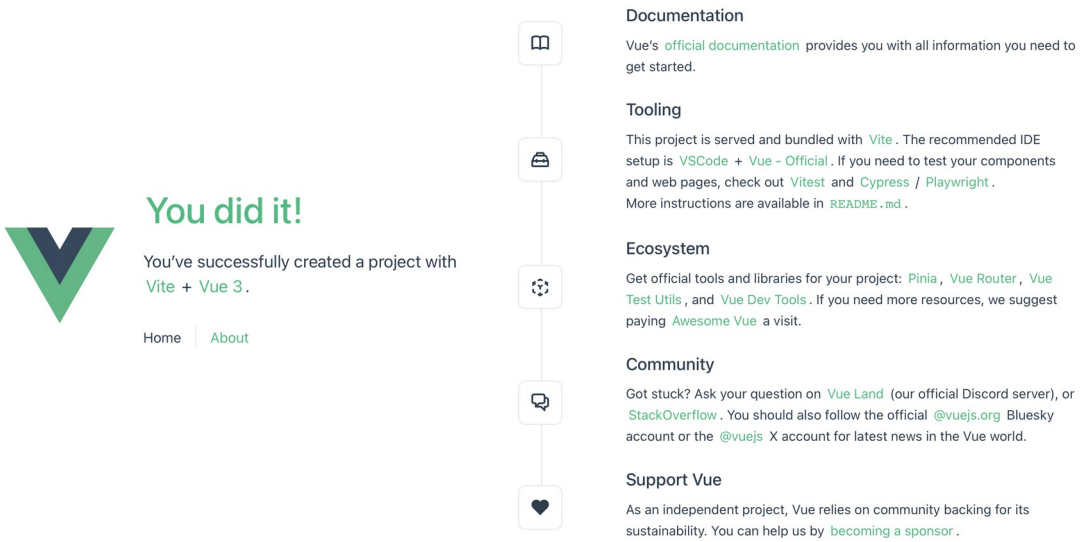


Рисунок 1. Стартовая страница созданного проекта Vue

Компоненты и работа с данными

Компоненты. Я разделил интерфейс на представление NotesPage.vue, карточку NoteCard.vue и обёртку BaseLayout.vue. Карточка получает заголовок name и текст text через props. Обёртка содержит slot и контейнер Bootstrap. На рисунке 2 показаны код карточки и структура проекта.

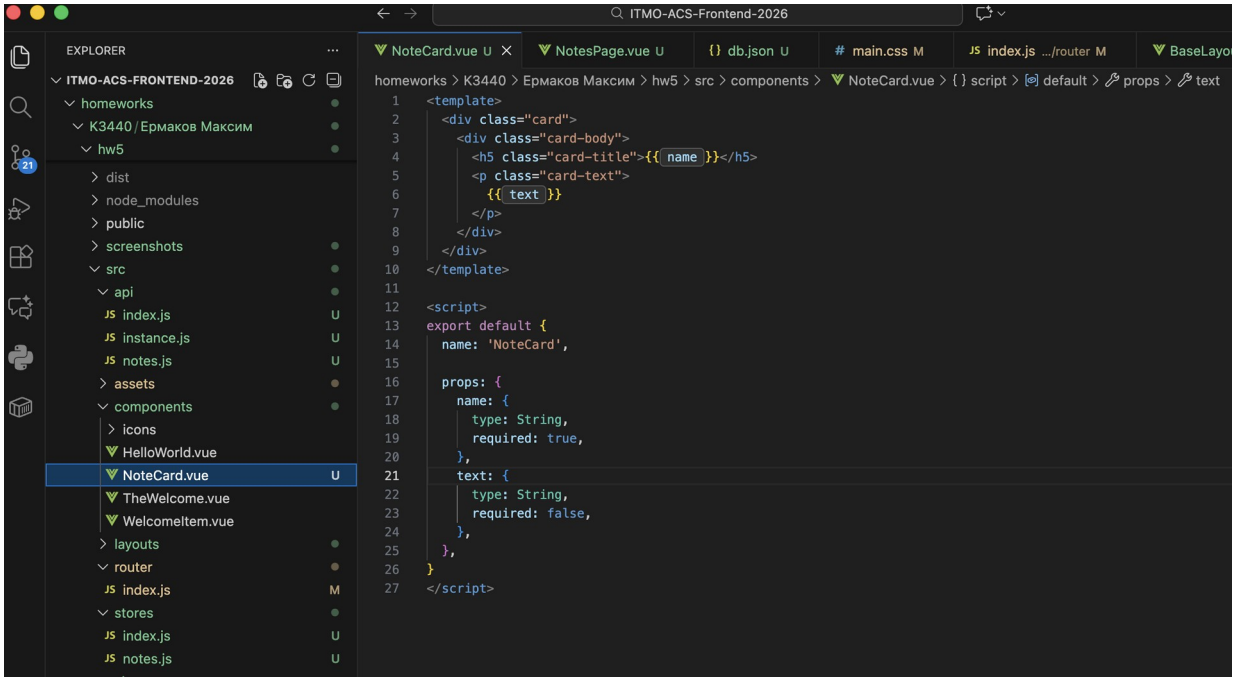


Рисунок 2. Компонент NoteCard и структура проекта

Маршрутизация. В src/router/index.js я связал адрес / с NotesPage.vue. Представление загружается через динамический import, а App.vue выводит его в router-view.

API и состояние. В src/api/instance.js я создал общий экземпляр Axios с базовым адресом http://localhost:3000. Класс NotesApi выполняет GET и POST /notes. Хранилище Pinia содержит массив notes и действия loadNotes и createNote. Плагин pinia-persists сохраняет состояние в localStorage. Для учебного API я запустил JSON Server с файлом db.json по примеру из ссылки в методичке.

Форма. Поля связаны с form.name и form.text через v-model. mapState даёт доступ к списку, mapActions - к действиям хранилища. В mounted я вызываю loadNotes; v-for выводит карточки с уникальным key. Обработчик @submit.prevent запускает создание заметки без перезагрузки страницы.

Проверка работы приложения

После загрузки страницы отобразилась первая заметка из базы. Затем я добавил ещё две записи. Метод createCard ожидает создание заметки, повторно загружает список и очищает поля формы вместе со значениями form.name и form.text. На рисунке 3 показан результат.

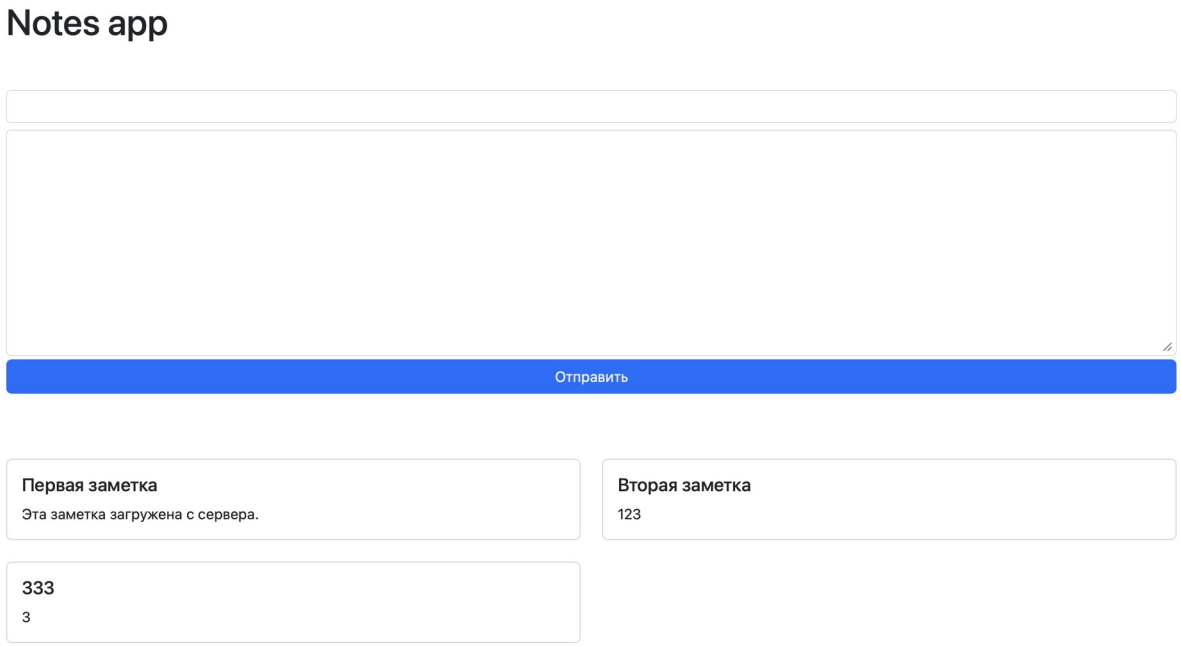


Рисунок 3. Приложение с тремя заметками и очищенной формой

Проверка API. В журнале сервера на рисунке 4 запросы POST /notes завершились кодом 201 Created, а последующие GET /notes - кодом 200 OK. Созданные записи сохранились в db.json.

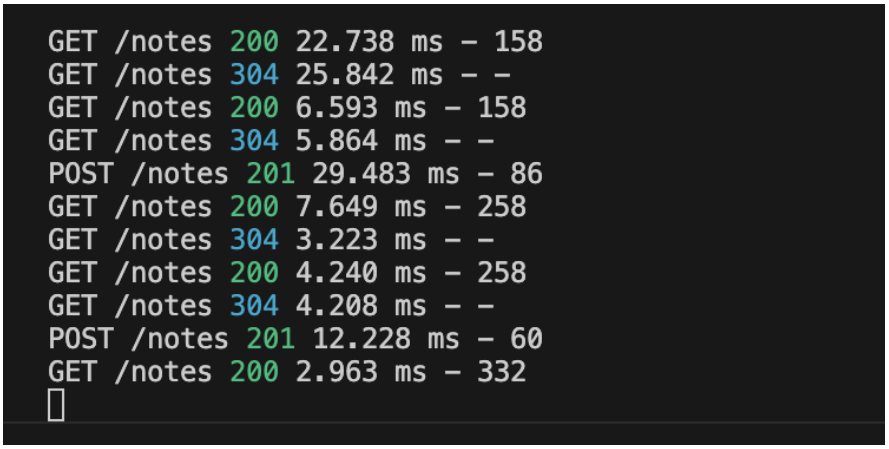


Рисунок 4. Успешные запросы получения и создания заметок

Вывод

Я освоил создание проекта и работу с зависимостями через npm, разделение интерфейса на компоненты и подключение маршрутизатора. Приложение получает и создаёт заметки через Axios, а Pinia связывает данные с интерфейсом. Исходный код приложения находится в папке hw5.